

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

for

Crypto Price Tracker

A Web-Based Application for Tracking Live Cryptocurrency Prices and Portfolios

Prepared By: KANISHK SAXENA

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for the Crypto Price Tracker, a web application that lets users monitor live cryptocurrency prices, follow a personal watchlist, and track a simple portfolio of holdings. It is meant to guide design, development and testing of the system.

1.2 Document Conventions

Functional requirements are identified using the format FR-XX. Each requirement uses the keyword "shall" to express a mandatory capability. Priority levels used are High (must have for a working system), Medium (important, can be simplified if time is short) and Low (optional / future work).

1.3 Intended Audience

- Developer(s) — to understand what to build.
- Evaluator / Reviewer — to check completeness and feasibility.
- UI/UX Designer — to design screens based on Section 3 and 4.
- Tester — to derive test cases from Section 3.

1.4 Project Scope

The Crypto Price Tracker will allow a user to register/login, view live prices for major cryptocurrencies, search and filter coins, add coins to a personal watchlist, and record manual buy/sell entries to track a simple portfolio with profit/loss. Price data will be pulled from a public market-data API. The application will show basic historical price charts for a selected coin.

The scope is kept intentionally small and realistic for a solo/small-team project. It is not a trading platform: it does not execute real trades, does not connect to exchange accounts, and does not handle real money. These are listed as future enhancements in Section 10.

1.5 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering – Requirements Engineering.
- CoinGecko / CoinMarketCap public API documentation.
- React.js, Node.js, Express.js and MongoDB official documentation.

2. Overall Description

2.1 Product Perspective

The Crypto Price Tracker is a new, self-contained web application. It follows a standard three-tier architecture (presentation, application/API, and data layer — see Section 8) and depends on a third-party market-data API for live price feeds.

2.2 Product Features (Summary)

- User registration, login and profile management.
- Live price listing for top cryptocurrencies with search and sort.
- Coin detail page with historical price chart.
- Personal watchlist (add/remove favourite coins).
- Manual portfolio tracking (add buy/sell entries, view profit/loss).
- Price alert notifications (email) when a coin crosses a target price.
- Basic dashboard summarising portfolio value and top movers.

2.3 User Classes and Characteristics

User Class	Description
Visitor	Unregistered user who can view live prices but cannot save a watchlist or portfolio.
Registered User	Logged-in user who manages a personal watchlist, portfolio and price alerts.
Administrator	Manages supported coins list, monitors API usage, and views basic app-wide stats.

2.4 Operating Environment

- Client side: any modern browser (Chrome, Firefox, Edge, Safari) on desktop or mobile.
- Server side: Node.js runtime, hosted on a cloud platform (e.g. Render, Railway) or run locally.
- Database: MongoDB Atlas (cloud) or a local MongoDB instance.
- External dependency: a public cryptocurrency market-data API (e.g. CoinGecko) for live prices.

2.5 Design and Implementation Constraints

- Must primarily use free / open-source technologies and a free-tier market-data API.
- Must be responsive and usable on both desktop and mobile screens.
- Subject to the rate limits of the free market-data API tier; prices may be cached briefly (e.g. 30–60 seconds) to stay within limits.
- No real money or exchange-account integration is included in this version.

2.6 Assumptions and Dependencies

- Users have basic familiarity with web forms and general cryptocurrency terms (coin, price, portfolio).
- Reliable internet connectivity is available at both client and server ends.
- The third-party price API remains available and free to use at the agreed request volume.
- Email delivery depends on a third-party SMTP service (e.g. Gmail SMTP / SendGrid free tier).

3. System Features (Functional Requirements)

This section lists the functional requirements grouped by module. Each requirement can be traced to a test case (Section 7) and a screen (Section 4).

3.1 Module: User Registration & Authentication

Description: Allows users to securely create accounts and log in.

ID	Requirement	Priority
FR-01	The system shall allow a new user to register using name, email and password.	High
FR-02	The system shall validate email format and enforce a minimum password strength on registration.	High
FR-03	The system shall authenticate registered users via email and password login.	High
FR-04	The system shall issue a secure session/token (e.g. JWT) upon successful login.	High
FR-05	The system shall allow a user to reset a forgotten password via a one-time email link/OTP.	Medium
FR-06	The system shall allow a logged-in user to log out, invalidating the active session.	Medium

3.2 Module: Live Price Listing

Description: Core price-viewing experience, available to visitors and registered users.

ID	Requirement	Priority
FR-07	The system shall display a list of top cryptocurrencies with current price, 24-hour change and market cap.	High
FR-08	The system shall allow the user to search for a coin by name or symbol.	High
FR-09	The system shall allow the user to sort the coin list by price, 24-hour change, or market cap.	Medium
FR-10	The system shall refresh displayed prices automatically at a fixed interval (e.g. every 60 seconds).	Medium
FR-11	The system shall show a coin detail page with a historical price chart (e.g. 24h / 7d / 30d ranges).	High

3.3 Module: Watchlist

Description: Lets a registered user track coins of personal interest.

ID	Requirement	Priority
FR-12	The system shall allow a registered user to add a coin to their personal watchlist.	High
FR-13	The system shall allow a registered user to remove a coin from their watchlist.	High
FR-14	The system shall display the user's watchlist with live prices on their dashboard.	High

3.4 Module: Portfolio Tracking

Description: Lets a registered user manually record holdings and track performance.

ID	Requirement	Priority
FR-15	The system shall allow a user to add a manual holding entry (coin, quantity, buy price, date).	High
FR-16	The system shall allow a user to edit or delete an existing holding entry.	Medium
FR-17	The system shall calculate and display the current value and profit/loss of each holding.	High
FR-18	The system shall display the total portfolio value and overall profit/loss on the dashboard.	High

3.5 Module: Price Alerts & Notifications

Description: Notifies users when a tracked coin crosses a target price.

ID	Requirement	Priority
FR-19	The system shall allow a user to set a target price alert (above/below) for a coin.	Medium
FR-20	The system shall send an email notification when a coin's price crosses the user's target.	Medium
FR-21	The system shall allow a user to view and delete their active price alerts.	Low

3.6 Module: Administrator Portal

Description: Basic configuration and monitoring for the app owner.

ID	Requirement	Priority
FR-22	The system shall allow the Admin to add or remove coins from the supported coin list.	Medium
FR-23	The system shall allow the Admin to view basic app usage statistics (registered users, active alerts).	Low

4. External Interface Requirements

4.1 User Interfaces

- Landing / Home Page — live price ticker, top movers, login/register call-to-action.
- Price List Page — searchable, sortable table/grid of coins with live prices.
- Coin Detail Page — price chart, key stats, add-to-watchlist and set-alert actions.
- User Dashboard — watchlist, portfolio summary, active alerts.
- Admin Panel — manage supported coins, view usage stats.
- All screens shall follow a consistent, responsive layout suitable for both desktop and mobile.

4.2 Hardware Interfaces

No specialized hardware is required. The system runs on standard client devices with an internet connection, and on any commodity server/cloud instance capable of running Node.js and MongoDB.

4.3 Software Interfaces

Component	Technology
Frontend	React.js (React Router, Axios, a charting library such as Chart.js/Recharts)
Backend / API	Node.js with Express.js — RESTful JSON APIs
Database	MongoDB (Mongoose ODM) — NoSQL document store
Authentication	JWT (JSON Web Tokens) with bcrypt password hashing
Market Data	Public crypto price API (e.g. CoinGecko) for live and historical prices
Email Service	SMTP service (e.g. Nodemailer with Gmail/SendGrid) for alert notifications
Hosting (suggested)	Frontend — Vercel/Netlify; Backend — Render/Railway; Database — MongoDB Atlas

4.4 Communication Interfaces

- All client-server communication shall occur over HTTPS using RESTful JSON APIs.
- Price data shall be retrieved from the market-data API over HTTPS, cached briefly to respect rate limits.

- Outbound alert emails shall be sent via the SMTP protocol through the configured email service.

5. Use Case Summary

Actor	Use Case	Brief Description
Visitor	View Live Prices	Browse the current price list without logging in.
User	Register / Login	Create an account or sign in to access personal features.
User	Search / Sort Coins	Find a coin by name, symbol, price or market cap.
User	Manage Watchlist	Add or remove coins from a personal watchlist.
User	Manage Portfolio	Record holdings and view profit/loss.
User	Set Price Alert	Get notified when a coin crosses a target price.
Admin	Manage Coin List	Add or remove coins available in the app.

5.1 Primary Flow — Adding a Coin to the Watchlist

1) User logs in → 2) Searches for a coin → 3) Opens the coin detail page → 4) Clicks "Add to Watchlist" → 5) System saves the coin against the user's account → 6) Coin appears on the user's dashboard with a live price.

6. Non-Functional Requirements

6.1 Performance Requirements

ID	Requirement	Priority
NFR-01	The system shall load any core page within 3 seconds under normal load on a standard broadband connection.	High
NFR-02	The system shall support at least 50–100 concurrent users without noticeable performance degradation.	Medium
NFR-03	Displayed prices shall be refreshed within 60 seconds of the source API updating.	Medium

6.2 Security Requirements

ID	Requirement	Priority
NFR-04	Passwords shall be stored using a one-way hashing algorithm (e.g. bcrypt); plain-text passwords shall never be stored.	High
NFR-05	All API endpoints that access personal data (watchlist, portfolio, alerts) shall require a valid authentication token.	High
NFR-06	All data in transit shall be encrypted using HTTPS/TLS.	High
NFR-07	User input shall be validated and sanitized on both client and server to prevent injection attacks.	High

6.3 Usability

- The interface shall be simple enough for a first-time user to find a coin's price in under 3 clicks/taps.
- The system shall be fully responsive across desktop, tablet and mobile screen sizes.
- Error and success messages shall be clear, specific, and shown close to the relevant form field/action.

6.4 Reliability & Availability

- The system shall target 99% uptime during normal operation.
- Watchlist and portfolio data shall be persisted in the database, not lost on a server restart.
- If the market-data API is temporarily unavailable, the system shall show the last cached prices with a stale-data indicator, rather than failing outright.

6.5 Maintainability & Portability

- The codebase shall follow a modular MVC-style structure (routes/controllers/models on the backend, reusable components on the frontend).
- The system shall run on any OS capable of hosting Node.js and MongoDB, and be accessible from any modern browser.

6.6 Scalability

Although scoped for a single market-data provider and a moderate user base in this version, the API and data model shall avoid provider-specific hard-coding where reasonably possible, so an additional data source or a paid API tier can be added later without a full redesign.

7. Software Testing Considerations

Each functional requirement in Section 3 shall map to at least one test case. Representative examples are given below; the full test case document can be maintained separately.

Test ID	Req.	Test Scenario / Expected Result
TC-01	FR-08	Search for a coin by symbol → matching coin(s) shown in the list.
TC-02	FR-12	Add a coin to the watchlist → coin appears on the user's dashboard.
TC-03	FR-17	Add a holding with a buy price below current price → profit shown as positive.
TC-04	FR-20	Set an alert below current price → email sent once price drops below target.
TC-05	FR-02	Register with a weak password → registration blocked with validation message.

Recommended testing types: Unit testing (backend controllers/validators using Jest/Mocha), Integration testing (API endpoints with Postman or Jest+Supertest), and Manual UI/UX testing across at least two browsers and one mobile viewport before release.

8. Proposed System Architecture

A MERN stack (MongoDB, Express.js, React.js, Node.js) three-tier architecture is recommended. This keeps the project fully JavaScript-based, is well documented, free to host on free tiers, and maps cleanly onto the module breakdown in Section 3.

- Presentation Layer — React.js SPA (price list, coin detail, dashboard, admin UI).
- Application Layer — Node.js + Express.js REST API (Auth, Prices, Watchlist, Portfolio, Alerts); calls the external market-data API.
- Data Layer — MongoDB (Users, Watchlists, Holdings, Alerts).

8.1 Suggested Technology Stack

Layer	Technology
Frontend	React.js, React Router, Axios, Chart.js/Recharts, Bootstrap/Tailwind CSS
Backend	Node.js, Express.js, JWT, bcrypt, Nodemailer, node-cron (for scheduled alert checks)
Database	MongoDB with Mongoose
External API	CoinGecko (or similar) public market-data API
Version Control	Git & GitHub
Testing	Postman (API testing), Jest (unit testing)
Deployment (suggested)	Vercel/Netlify (frontend), Render/Railway (backend), MongoDB Atlas (database)

9. Data Model Overview (for Database Planning)

9.1 Core Entities

Entity	Key Attributes
User	<u>id</u> , name, email, passwordHash, role (user/admin), createdAt
Coin	<u>id</u> , symbol, name, currentPrice, marketCap, lastUpdated
WatchlistItem	id, userId, coinId, addedAt
Holding	id, userId, coinId, quantity, buyPrice, buyDate
PriceAlert	<u>id</u> , userId, coinId, targetPrice, direction (above/below), isActive, createdAt
Notification	id, userId, message, type, isRead, createdAt

9.2 Key Relationships

- A User has many WatchlistItems, Holdings and PriceAlerts (one-to-many, each).
- A Coin can appear in many WatchlistItems, Holdings and PriceAlerts (one-to-many).
- A User can receive many Notifications (one-to-many).

10. Future Enhancements (Out of Current Scope)

- Exchange account integration for automatic portfolio sync (read-only API keys).
- Push/SMS notifications alongside email.
- Support for multiple currencies (USD, EUR, INR, etc.) for displayed values.
- Social features: sharing a watchlist or portfolio performance publicly.
- AI-based price trend summaries or basic technical indicators.
- Native mobile app (iOS/Android).

11. Glossary

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer — a web API architecture style

Term	Definition
JWT	JSON Web Token — used for stateless authentication
MERN	MongoDB, Express.js, React.js, Node.js — a full JavaScript web stack
Watchlist	A user's saved list of coins they want to monitor
Portfolio	A user's recorded set of cryptocurrency holdings
Market Cap	Total market value of a cryptocurrency (price × circulating supply)

End of Document — Crypto Price Tracker SRS v1.0